SYSTEM GUIDE · IMPLEMENTATION REFERENCE

CAV Photo Studio and Cafe

Project overview, architecture, operating guide, data model, API surface, security, and deployment instructions

> A source-based guide to the application as implemented in this repository. Prepared 20 October 2026.

React and Capacitor client · Django REST API · PostgreSQL database
Repository documentation

Contents

1. Project overview and scope
2. System architecture and technology stack
3. Features and application modules
4. Roles and permissions
5. Database schema and relationships
6. Key workflows
7. API reference and integrations
8. Authentication and security
9. Setup, environment, and deployment
10. Testing and operational checks
11. Troubleshooting and source map

> Optional integrations require credentials, feature flags, external provider availability, and deployment configuration.

1. Project overview and scope

CAV Photo Studio and Cafe is a web application for a combined photography studio and cafe. It provides a customer-facing site and booking experience alongside authenticated customer, staff, and administrator workspaces. Its Django REST API supports bookings, payments, cafe sales, stock, notifications, analytics, and sales forecasting.

The codebase packages the React client as an Android app through Capacitor. A local print bridge supports receipt printing from a staff workstation. Deployed services are configured in `render.yaml`.

Objectives reflected in the implementation

- Present studio services, packages, cafe menu, gallery, story, booking instructions, and privacy information.
- Let customers manage profiles, appointments, payments, reviews, and notifications.
- Support staff workflows for bookings, POS, payment verification, inventory, and customer support.
- Give administrators account management, analytics, end-of-day reporting, unavailable dates, and controlled system reset.
- Provide optional Google sign-in, email/SMS, browser push, receipt OCR, and grounded chatbot generation.

> **Scope.** This guide follows code and checked-in deployment files. It does not assume optional providers are configured or hosted services are available. The older `tech_stack_architecture.md` is a proposal, not a current implementation specification.

2. System architecture and technology stack

Request path

```
Browser / Android WebView
 React 19 client + React Router + Axios
       | HTTPS JSON API; cookie credentials and CSRF for writes
       v
 Django 5.2 + Django REST Framework
 users | booking | payment | pos | inventory | dashboard
 forecasting | chatbot | notifications | audit | gallery
       | Django ORM
       v
 PostgreSQL (DATABASE_URL; Supabase-compatible)

Optional: Google OAuth, iProgSMS, SMTP/Resend, Web Push,
OCR.Space/Tesseract, OpenRouter
Local workstation: React build + Node print bridge
```

Frontend and API are separate deployable services. API URL selection in `frontend/src/api/config.js`: browser builds use `REACT_APP_API_BASE_URL`; local browser defaults to `http://localhost:8000`; production/native defaults to the public Render API unless overridden.

Technology stack

Area	Implemented technology
Web client	React 19, Create React App/react-scripts 5, React Router 7, Axios, Tailwind CSS 3, Recharts, Lucide React.
Mobile	Capacitor 8 Android; Google social login plugin enabled in Android configuration.
API	Python 3.11 image, Django 5.2, Django REST Framework, SimpleJWT, django-cors-headers, Gunicorn.
Database	Django ORM and PostgreSQL URL config (Supabase compatible); psycopg2-binary. CI uses SQLite.
Data/OCR	pandas, NumPy, scikit-learn, SciPy, joblib, Pillow, pytesseract; Docker installs Tesseract.
Deployment/QA	Render static frontend and Docker API; GitHub Actions, Jest/RTL, Django tests, pip-audit, Bandit, npm audit, axe-core and secret scan.

Repository map

Path	Responsibility
`frontend/src`	React routes/pages, auth context, API client, UI, utilities.
`backend/config`	Django settings, URL table, security middleware, health, WSGI/ASGI.
`backend/apps`	Domain apps, REST views and models.
`backend/ml`, `ml/training`	Forecast engine, historical repository and training support.
`android`, `frontend/android`	Android Gradle projects; root Capacitor config points to `android`.
`render.yaml`, `.github/workflows/quality.yml`	Deployment and CI definitions.

3. Features and application modules

Module	Implemented behavior
Public site	Home, studio, cafe menu, gallery, story, booking guide, login, privacy policy.
Accounts	Registration/OTP, login, OTP password reset, Google auth, profile editing, staff accounts, privacy consent/requests/export.
Bookings	Service/package list, availability, booking CRUD, customer edit window, admin unavailable dates, reviews, payment and receipt verification.
POS	Walk-in and booking-linked orders, discounts, payment, idempotency, receipt data/print state, end-of-day reports/reprints.
Inventory	Products/ingredients, categories, suppliers, purchase orders, stock adjustments, recipes, unit conversion, expiry/stock alerts.
Notifications	In-app notifications/read state, browser push subscriptions, delivery records, booking reminders and delivery command.
Dashboard	Revenue/transaction measures, booking counts, history chart, stock alerts, recent activity, top items/packages.
Forecasting	Prediction endpoint and CSV history import with batch/record tracking; staff/admin access.
Chatbot	Public query, local knowledge/FAQ and booking assistant, optional OpenRouter generation; admin FAQ management.
Audit/gallery	Admin runtime-error intake; gallery endpoint serves constants, not database records.

Frontend routes in `frontend/src/App.js`: `/`, `/photo-studio`, `/cafe-menu`, `/gallery`, `/our-story`, `/how-to-book`, `/login`, `/privacy-policy`, and protected `/customer`, `/staff`, `/admin`.

> Dashboard metrics are calculated from bookings, orders, payments, inventory. Sales/reports apps define no summary models. Gallery and chatbot FAQ are not database backed in current code.

4. Roles and permissions

Roles are `CUSTOMER`, `STAFF`, and `ADMIN`. Backend permissions and view checks are authoritative; frontend route guards only control navigation.

Role	Observed access
Anonymous	Public pages; service/package/payment-method listings; gallery; chatbot query; auth flows; push config key.
Customer	Own profile, bookings, payment submission and owned receipt access, reviews, own notifications/subscriptions, privacy consent/requests/export, customer chatbot context.
Staff	Booking/payment operations; POS; inventory/purchase; analytics and forecasting; staff chatbot context. Account admin, system reset, unavailable dates, end-of-day reports, FAQ CRUD and delivery inspection are admin-only.
Admin	Staff capabilities plus user management, reset, unavailable dates, end-of-day reports/reprints, FAQ CRUD, review listing, delivery inspection and runtime error review.

`IsStaffOrAdmin` protects POS, inventory, analytics and forecasting. `IsAdmin` protects user administration, FAQs, unavailable dates, end-of-day reports, runtime error intake and delivery records. Booking/payment views also filter by owner and role.

5. Database schema and relationships

Django migrations manage the schema. Production requires PostgreSQL via `DATABASE_URL` or `SUPABASE_DATABASE_URL`; CI uses SQLite. This summary omits inherited Django auth fields.

Domain/model	Important data and relationships
`CustomUser`	AbstractUser plus role, registration method, phone, address, profile photo. Case-insensitive unique username and nonblank email/phone.
`Customer`	One-to-one user profile with points, birthdate, tier, notes.
Privacy/reset	`PrivacyConsent` type/version/grant/withdrawal; `PrivacyRequest` type/status; `PasswordResetOTP` hashed OTP/token, expiry, attempts, timestamps.
Booking catalog	`Service` has many `Package` rows; case-insensitive name uniqueness.
Booking	Customer/package, date/time/status, notes, JSON items/change history, idempotency. Active slot unique for pending/confirmed statuses. `BookingReview`, `StudioUnavailableDate`, `BookingDateLock`.
Inventory	`Category`, `Supplier`, `Product` (product/ingredient, stock, units, prices, dates, recipe JSON) and `InventoryEvent` movements/purchases with JSON payload.
POS	`Order` stores staff, optional booking, subtotal/discount/total, payment status/type, JSON lines/reports/print state and transaction/idempotency keys.
Payments	`Payment` belongs to exactly one booking or order; amount, method, reference, receipt, status, verifier/time, metadata, idempotency. Uniqueness constraints protect references and idempotency.
Notifications	`Notification` user/optional booking; `PushSubscription` endpoint/keys; `NotificationDelivery` channel/provider state, attempts, errors, retries.
Forecast/audit	`SalesHistoryImport` batch; `SalesHistoryRecord` imported rows/amounts. `AuditLog` actor/action/metadata/IP/time; model blocks deletion.

Relationship overview

```
CustomUser 1--1 Customer
CustomUser 1--many Booking; Booking many--1 Package many--1 Service
Booking 1--many Payment; Booking 1--many linked Order; Booking 1--0/1 Review
Order 1--many Payment; Order staff -> CustomUser
Product -> inventory events; Category/Supplier references are integer fields
CustomUser 1--many Notification -> optional Booking
Notification 1--many Delivery; CustomUser 1--many PushSubscription
SalesHistoryImport 1--many SalesHistoryRecord
```

Product category/supplier fields are integer IDs with denormalized details, not Django foreign keys. POS lines and report snapshots are JSON rather than normalized order-item tables.

6. Key workflows

Registration and sign-in

1. Registration creates a customer account and starts contact verification through the configured delivery path.
2. OTP verification completes registration. Password reset verifies an OTP, then confirms with a short-lived reset token.
3. Admin sign-in has an additional OTP/MFA step sent using configured SMS credentials.
4. Login sets access/refresh cookies. The client obtains CSRF before writes, refreshes expired sessions, and redirects to login if refresh fails.

Booking and payment

1. Customer checks availability and submits a booking. Active slot uniqueness and date-level locking protect concurrent requests.
2. Customers see their own bookings. Pending bookings may be edited; confirmed bookings have a 24-hour pre-appointment customer edit deadline.
3. Customer submits payment method/reference/receipt. Staff/admin verify payment; configured QR methods are surfaced by API.
4. Booking events create in-app notifications and may dispatch email/SMS/push when enabled.

POS and close of day

1. Staff create walk-in or booking-linked order. Idempotency key limits duplicate submissions; server stores item snapshots and payment state.
2. Payment links to order. Printing uses staff client/local bridge; print state can be stored.
3. Admin reviews day summary, creates end-of-day report, and can reprint/mark printed.

Forecasting

Staff/admin import historical CSV. Batches record source hash, accepted/skipped rows, date range. Source can be the included cafe/sparkling-drinks CSV or `SALES_HISTORY_CSV_PATH`. Forecasts are computed in `backend/ml`; predictions are not stored in dedicated prediction tables.

7. API reference and integrations

Paths are relative to the API host. JSON is default. View/serializer code is authoritative for methods, payload fields and response details.

Group	Routes	Typical access
Health/admin	`GET /health/`, Django `/admin/`	Health public; Django admin uses staff permission.
Auth/privacy	`/api/auth/login/`, `/login/mfa/resend/`, `/csrf/`, `/token/refresh/`, `/logout/`, `/register/`, `/register/verify/`, `/register/resend/`, `/google/`, `/profile/`, `/forgot-password/` plus `/verify/`, `/confirm/`; `/api/privacy/consents/`, `/requests/`, `/export/`; `/api/auth/users/` and id; `/api/admin/system-reset/`	Public auth steps; authenticated self-service; admin actions.
Bookings	`/api/bookings/services/`, `/packages/`, `/availability/`, `/payment-methods/`, `/studio-unavailable-dates/` and id, `/api/bookings/` and id, `/reviews/`, `/reviews/all/`, `/payments/`, payment id `/receipt/`, `/ocr/`, `/reference-check/`, id `/verify/`	Catalog/payment methods public; booking/payment access depends on owner and role.
Inventory	`/api/inventory/products/` and id; `ingredients/` and id; `ingredient-movements/`; `recipes/`, `recipes/generate/`; `movements/`; `categories/` and id; `suppliers/` and id; `purchase-orders/` and id	Staff/admin operations; limited public product listing.
POS/report	`/api/pos/orders/` and id; `/payments/`; `/end-of-day-summary/`; `/end-of-day-reports/`, report id `/reprint/`, `/mark-printed/`	Staff/admin POS; admin reports.
Analytics/support	`/api/dashboard/analytics/`; `/api/forecasting/predictions/`; `/api/forecasting/history/import/`; `/api/chatbot/query/`; `/api/chatbot/faqs/` and id; `/api/audit/client-runtime-errors/`	Analytics/forecast staff/admin; chatbot query public; FAQ admin.
Notifications/gallery	`/api/notifications/`, `/read-all/`, id `/read/`, `/push/config/`, `/push/subscription/`, `/deliveries/`; `/api/gallery/images/`	Own notifications/subscriptions authenticated; delivery list admin; gallery public.

Integrations

Service	Implementation/condition
Google	Frontend obtains identity credential; backend validates issuer/audience against `GOOGLE_CLIENT_ID`. Native config enables Google only.
iProgSMS	OTP/admin verification and notifications; notification dispatch has a separate enable flag.
Email	SMTP/Django email or Resend HTTP API. Console backend and delivery-off defaults.
Web Push	pywebpush and VAPID; service worker `frontend/public/push-sw.js`; optional.
OCR	OCR.Space default and Tesseract fallback, installed in Docker. Key stays backend-side.
Chat	Optional OpenRouter. Local code supplies grounded context and live booking/pricing logic.
Printing	Local Node bridge, default port 8765. Requires workstation and printer.
Payments	Cash, GCash, Maya, MariBank recorded as methods. QR assets/config exist; no payment gateway or automatic settlement is implemented.
Gallery	Includes local static photo and one Unsplash-hosted cafe image.

8. Authentication and security

- Custom Cookie JWT authenticator. Access lifetime defaults to 15 minutes; refresh lifetime 7 days, with rotation and blacklist after rotation.
- Browser uses HttpOnly access/refresh cookies. Writes fetch CSRF token and send credentials/header.
- Cross-origin production cookies default secure and SameSite=None; CORS allows credentials and configured origins.
- Password checks include similarity, minimum eight characters, custom strength, common and numeric password validators. OTP expiry, attempts, resend controls and throttles are configured.
- DRF default is authenticated unless a view allows anonymous access. Throttles cover anonymous/user/auth/password reset/chatbot/OCR/sensitive scopes.
- Security settings include HTTPS redirect, HSTS, secure cookies, content-type protection, frame denial, referrer policy, CSP and permissions policy. Render static frontend sets response headers.
- Upload limits/path validation exist. Audit log deletion is blocked at model/queryset level.
- Secrets come from environment; `backend/.env.example` is a template. CI runs `scripts/secret_scan.py`.

Set matching Google IDs, correct CORS/CSRF origins, HTTPS URLs, DB credentials and provider secrets only on backend. System reset is destructive and admin/password guarded.

9. Setup, environment, and deployment

Prerequisites and local setup

Root package declares Node >=22 and npm >=10. Backend Docker uses Python 3.11. PostgreSQL is required by default settings. Tesseract is optional locally and installed in Docker.

1. Run `npm ci` from repository root.
2. Create `backend/.env` from `backend/.env.example`. Set `DJANGO_DEBUG=true`, development secret, PostgreSQL `DATABASE_URL`, allowed hosts, local CORS/CSRF origins. SQLite is used in CI, not as documented production DB.
3. In `backend`, create Python 3.11 virtual environment, install `requirements.txt`, run `python manage.py migrate`, then `python manage.py runserver` (port 8000).
4. Create `frontend/.env` from `frontend/.env.example`, set API to `http://localhost:8000`, then run `npm start` in `frontend` (CRA port 3000).
5. Provide runtime passwords before using the seed account command. No default credentials should be assumed.

`npm run cap:sync` builds/verifies web assets then syncs Android. `npm run cap:run:android` syncs and launches target.

Environment variables

Group	Variables and notes
Core	`DJANGO_SECRET_KEY` (required with debug off; fallback `SECRET_KEY`), `DJANGO_DEBUG` (false), `DJANGO_ALLOWED_HOSTS`, `DATABASE_URL` or `SUPABASE_DATABASE_URL` (required), `DATABASE_SSL_REQUIRE` (true for PostgreSQL), `TIME_ZONE` (Asia/Manila), `MEDIA_ROOT`.
Frontend/printing	`REACT_APP_API_BASE_URL`, `REACT_APP_MOBILE_API_BASE_URL`, `REACT_APP_GOOGLE_CLIENT_ID`, `REACT_APP_LOCAL_PRINT_BRIDGE_URL`, `LOCAL_PRINT_BRIDGE_PORT`.
Origins/cookies	`CORS_ALLOWED_ORIGINS`, `CSRF_TRUSTED_ORIGINS`, `CORS_ALLOW_CREDENTIALS`, `SECURE_SSL_REDIRECT`, `SECURE_HSTS_SECONDS`, `SECURE_HSTS_PRELOAD`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `AUTH_COOKIE_SECURE`, `AUTH_COOKIE_SAMESITE`, `CSRF_COOKIE_SAMESITE`, `AUTH_COOKIE_DOMAIN`, `AUTH_ACCESS_COOKIE_NAME`, `AUTH_REFRESH_COOKIE_NAME`, `SESSION_COOKIE_AGE`.
Auth/privacy	`GOOGLE_CLIENT_ID`, `JWT_ACCESS_MINUTES`, `ADMIN_MFA_TTL_MINUTES`, `ADMIN_MFA_MAX_ATTEMPTS`, `PASSWORD_RESET_OTP_TTL_MINUTES`, `PASSWORD_RESET_OTP_MAX_ATTEMPTS`, `PASSWORD_RESET_TOKEN_TTL_MINUTES`, `PASSWORD_RESET_RESEND_COOLDOWN_SECONDS`, `PASSWORD_RESET_MAX_REQUESTS_PER_HOUR`, `PRIVACY_REQUEST_RETENTION_DAYS`, `PASSWORD_RESET_RETENTION_DAYS`; seed: `SEED_ADMIN_USERNAME`, `SEED_ADMIN_PHONE`, `SEED_ADMIN_PASSWORD`, `SEED_STAFF_PASSWORD`, `SEED_CUSTOMER_PASSWORD`.
Email/SMS/push	`EMAIL_PROVIDER`, `EMAIL_BACKEND`, `DEFAULT_FROM_EMAIL`, `EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `EMAIL_USE_TLS`, `EMAIL_USE_SSL`, `EMAIL_TIMEOUT`, `EMAIL_DELIVERY_ENABLED`, `RESEND_API_KEY`, `RESEND_API_URL`, `SMS_DELIVERY_ENABLED`, `IPROG_SMS_API_TOKEN`, `IPROG_SMS_API_BASE_URL`, `IPROG_SMS_API_TIMEOUT`, `SMS_MAX_ATTEMPTS`, `SMS_RETRY_DELAY_SECONDS`, `BOOKING_REMINDER_WINDOW_MINUTES`, `PUSH_DELIVERY_ENABLED`, `PUSH_VAPID_PUBLIC_KEY`, `PUSH_VAPID_PRIVATE_KEY`, `PUSH_VAPID_SUBJECT`, `PUSH_DELIVERY_TTL_SECONDS`.
OCR/chat	`OCR_API_PROVIDER`, `OCR_API_URL`, `OCR_API_KEY`, `OCR_API_TIMEOUT`, `OCR_SPACE_ENGINE`, `OCR_FALLBACK_TESSERACT`, `TESSERACT_CMD`; `OPENROUTER_API_KEY`, `OPENROUTER_API_URL`, `OPENROUTER_MODEL`, `OPENROUTER_TIMEOUT_SECONDS`, `OPENROUTER_MAX_TOKENS`, `OPENROUTER_SITE_URL`, `OPENROUTER_APP_NAME`.
Data/limits	`SALES_HISTORY_CSV_PATH`, `DATA_UPLOAD_MAX_MEMORY_SIZE`, `FILE_UPLOAD_MAX_MEMORY_SIZE`, `DATA_UPLOAD_MAX_NUMBER_FIELDS`, `DRF_ANON_THROTTLE_RATE`, `DRF_USER_THROTTLE_RATE`, `DRF_AUTH_THROTTLE_RATE`, `DRF_PASSWORD_RESET_THROTTLE_RATE`, `DRF_CHATBOT_THROTTLE_RATE`, `DRF_OCR_THROTTLE_RATE`, `DRF_SENSITIVE_THROTTLE_RATE`, `CONTENT_SECURITY_POLICY`, `PERMISSIONS_POLICY`, `APP_LOG_LEVEL`, `PAYMENT_QR_GCASH_URL`, `PAYMENT_QR_MAYA_URL`, `PAYMENT_QR_MARIBANK_URL`.

Render and Android

`render.yaml` defines static frontend build `npm ci && npm run build` from `frontend/build`, with SPA rewrite. Backend Docker installs requirements/Tesseract, collects static files, runs migrations/check, then Gunicorn on `$PORT`. Configure secrets and persistent media storage separately. Back up production DB before migrations. Android uses ID `com.cav.photostudio`; mobile API URL must be public HTTPS and is embedded at build time.

10. Testing and operational checks

GitHub Actions `.github/workflows/quality.yml` runs on pushes and pull requests.

Job	Checks
Backend	Python 3.11; install requirements; Django `check --deploy`; migration drift/apply/check; tests for audit, booking, chatbot, dashboard, forecasting, gallery, inventory, notifications, POS, reports, sales and users; pip-audit; Bandit.
Frontend	Node 20; `npm ci`; Jest noninteractive/in-band; build; npm audit; serve build and run axe-core CLI.
Secrets	`scripts/secret_scan.py`.

```
cd backend
python manage.py check --deploy
python manage.py makemigrations --check --dry-run
python manage.py migrate --noinput
python manage.py test audit booking chatbot dashboard forecasting gallery inventory notifications pos reports
sales users

cd frontend
npm test -- --watchAll=false --runInBand
npm run build
```

Check API with `GET /health/`. Inspect Render/Django logs and browser network/console. Management commands include `process_notification_deliveries` and `send_booking_reminders`; repository does not declare a Render cron service.

11. Troubleshooting and source map

Symptom	Checks
API fails at startup	Set `DJANGO_SECRET_KEY` with debug off and `DATABASE_URL`; verify PostgreSQL credentials/SSL and allowed hosts.
CORS/CSRF or API connection	Check built API URL and exact CORS/CSRF origin. Cookie auth needs credentials and proper secure/SameSite settings.
Login loop	Inspect cookies, cross-site policy, refresh response, HTTPS, frontend/API host relation and blacklist migrations.
Google auth rejected	Match frontend/backend Google IDs; configure allowed JavaScript origins and redirect settings.
OTP/reset missing	Check destination, provider credentials, enable flag, cooldown/rate limit, sender and delivery records. Console backend only logs email.
Push not delivered	Enable delivery and configure all VAPID values; check secure origin, permission/service worker and delivery status.
OCR fails	Check provider/key/timeout, file type/size, external API and Tesseract binary/language.
Printer unavailable	Start bridge; verify URL/port 8765, local reachability, printer driver/name and browser restrictions.
Build/tests fail	Use declared Node/npm locally, run `npm ci` at root, Python 3.11/backend requirements; inspect CI step/migration state.
Forecast has no history	Check CSV format/date coverage, import counts and `SALES_HISTORY_CSV_PATH`.

Primary implementation files

- `backend/config/settings.py`, `backend/config/urls.py`, `backend/config/middleware.py`
- `backend/apps/*/models.py`, views, serializers, services and migrations
- `frontend/src/App.js`, `context/AuthContext.jsx`, `api/client.js`, `api/config.js`, pages
- `frontend/local-print-bridge.js`, `frontend/src/utils/localPrinting.js`
- `backend/.env.example`, `frontend/.env.example`, `backend/Dockerfile`, `render.yaml`, CI workflow

Documents repository code and configuration inspected on 20 October 2026. Provider behavior, production values and hosted service state depend on deployment configuration.